Day 1 – Problem Solving Techniques and Data Structures I

Set 1: Retail Order Desk – Practical Record

Student details

Field	**Fill in before submission**
Student name	Kuchi Yasaswini
Roll number	-
Class / batch	Batch 1
Date	29-10-2026

Aim

Solve the five Retail Order Desk questions using real order data: search by order ID, sort totals, check possible repeated orders, model undo and dispatch, and calculate change.

What is included

Each question includes the algorithm, Python code, a worked example, a dry run, and the result and complexity.

Set 1 questions

1. Search an order by orderId
2. Sort order amounts in descending order
3. Check same-customer, same-amount orders from the same minute
4. Use a stack for undo and a queue for dispatch
5. Compare greedy change with dynamic programming

.

Ten sample orders from the real dataset

The prepared orders.csv has 96,478 delivered orders. The ten rows below are from that file. The search and sorting tests use real rows from the same data.

Field in orders.csv	Source field / definition
orderId	orders.order_id
customerId	customers.customer_unique_id (stable customer key)
amount	Sum of items.price + items.freight_value for each orderId; BRL
createdAt	orders.order_purchase_timestamp; timestamp as stored

Sample records (shaded rows match under Question 3)

#	orderId	customerId	amount (BRL)	createdAt
1	246b14cc444e8a7df0afd80495dfa052	031e19fc630c4121f1238716f41675c3	R$17.97	2018-07-04 01:20:01
2	53cdb2fc8bc7dce0b6741e2150273451	af07308b275d755c9edb36a90c618231	R$141.46	2018-07-24 20:41:37
3	f014aa85d8f495d824d120e9035faacb	050555c26479bbb8ba0740c37e7454d0	R$49.44	2018-04-07 12:07:04
4	47770eb9100c2d0c44946d9cf07ec65d	3a653a41f6f9fc3d2a113cf8398680e8	R$179.12	2018-08-08 08:38:49
5	f707ff2a9514dce20b267136b60af042	031e19fc630c4121f1238716f41675c3	R$17.97	2018-07-04 01:20:01
6	4f173306347cca2ee2a05590d617854a	050555c26479bbb8ba0740c37e7454d0	R$49.44	2018-04-07 12:07:04
7	56ae029ed8bd31758f66831a64bf5629	06a52782a04f0086d16b9c22d0e29438	R$51.11	2017-05-06 20:11:10
8	d9e44c3fd2ce16086619f299e92e12d8	06a52782a04f0086d16b9c22d0e29438	R$51.11	2017-05-06 20:11:10
9	84f5e6c0a0e3155e38c00f434ba90ce8	06a52782a04f0086d16b9c22d0e29438	R$51.11	2017-05-06 20:11:11
10	e481f51cbdc54678b7cc49136f2d6af7	7c396fd4830fd04220f754e42b4e5bff	R$38.71	2017-10-02 10:56:33

Rows 1 and 5, rows 3 and 6, and rows 7–9 have the same customer, amount, and purchase minute. Row 10 is used for search; row 4 has the largest total in this sample.

Question 1

Task 1 – Linear and binary search

Aim: Find an order by orderId. Compare a linear scan with binary search after sorting the IDs.

Pseudocode

```
LINEAR SEARCH(orders, targetId)
    FOR each order from first to last
        comparisons ← comparisons + 1
        IF order.orderId = targetId: RETURN order, comparisons
    RETURN NOT_FOUND, comparisons

BINARY SEARCH(sortedOrders, targetId)
    low ← 0; high ← LENGTH(sortedOrders) − 1
    WHILE low ≤ high
        mid ← FLOOR((low + high) / 2)
        comparisons ← comparisons + 1
        IF sortedOrders[mid].orderId = targetId: RETURN sortedOrders[mid]
        IF sortedOrders[mid].orderId < targetId: low ← mid + 1
        ELSE: high ← mid − 1
    RETURN NOT_FOUND
```

Python code — core functions

```
import csv
with open("orders.csv", newline="", encoding="utf-8") as f:
    orders = list(csv.DictReader(f))

def linear_search(rows, target_id):
    comparisons = 0
    for order in rows:
        comparisons += 1
        if order["orderId"] == target_id:
            return order, comparisons
    return None, comparisons

def binary_search(sorted_rows, target_id):
    low, high, comparisons = 0, len(sorted_rows) - 1, 0
    while low <= high:
        mid = (low + high) // 2
        comparisons += 1
        key = sorted_rows[mid]["orderId"]
        if key == target_id: return sorted_rows[mid], comparisons
        if key < target_id: low = mid + 1
        else: high = mid - 1
    return None, comparisons

sorted_orders = sorted(orders, key=lambda x: x["orderId"])
```

Sample run — Question 1

Search the ten-row sample for orderId e481f51cbdc54678b7cc49136f2d6af7. It is at row 10 in the sample.

Program output

```
TASK 1 — SEARCH REAL ORDERS
Sample orderId: e481f51cbdc54678b7cc49136f2d6af7.
On the 10-row sample: linear found=True in 10 comparisons; binary found=True in 2.
Benchmark: 50,000 delivered records, one row per orderId; target at row 49955.
Linear: 49,955 comparisons; median 3.611 ms/lookup.
Binary: 12 comparisons; median 0.0024 ms/lookup after IDs are sorted.
One-time sorting of 50,000 orderIds: median 17.963 ms.
Each row is one actual order; sorting is excluded from the binary lookup time.
```

Screenshot: output from the search program.

Dry run A — linear scan of the ten sample rows

Check	orderId suffix	Comparison	Action
1	`95dfa052`	not equal	continue
2	`50273451`	not equal	continue
3	`035faacb`	not equal	continue
4	`f07ec65d`	not equal	continue
5	`b60af042`	not equal	continue
6	`d617854a`	not equal	continue
7	`64bf5629`	not equal	continue
8	`e92e12d8`	not equal	continue
9	`4ba90ce8`	not equal	continue
10	`6f2d6af7`	equal	FOUND

The target occurs at sample position 10, so this scan makes 10 ID comparisons.

Dry run B — binary search on sorted sample IDs

Step	low	high	mid	orderId at mid	Comparison	Action
1	0	9	4	`56ae029e…bf5629`	56ae < e481	low ← 5
2	5	9	7	`e481f51c…2d6af7`	equal	FOUND

After sorting the ten IDs, binary search reaches the target in two midpoint checks.

Results — Question 1

I tested the first 50,000 delivered rows. The timed target is at row 49,955 in the original list; the binary-search copy is sorted first.

Method	Comparisons	Median time per lookup	Notes
Linear search	49,955	3.611 ms	Original row order
Binary search	12	0.0024 ms	Sorted IDs; lookup only
Sort 50,000 IDs	—	17.963 ms	One-time preprocessing

Search checks at different positions

Probe	Linear checks	Binary checks	Interpretation
First row match	1	15	Linear stops immediately
Middle row match (25,000)	25,000	14	Binary halves the search range
Last row match (50,000)	50,000	12	Linear scans almost the full file
Missing ID*	50,000	15	Both must establish absence

Linear search: O(n) time and O(1) extra space. Binary search: O(log n) time and O(1) extra space.

Question 2

Task 2 – Sorting order amounts

Aim: Sort order amounts from largest to smallest. Trace the first three bubble-sort passes and compare bubble sort and merge sort with Python sorted() on 10,000 real totals.

Pseudocode

```
BUBBLE SORT DESCENDING(values)
    a ← COPY(values)
    FOR pass ← 0 TO LENGTH(a) − 2
        swapped ← FALSE
        FOR j ← 0 TO LENGTH(a) − pass − 2
            IF a[j] < a[j + 1]
                SWAP a[j], a[j + 1]; swapped ← TRUE
        IF swapped = FALSE: BREAK
    RETURN a

MERGE SORT DESCENDING(values)
    IF LENGTH(values) ≤ 1: RETURN values
    split values into left and right halves
    left ← MERGE SORT DESCENDING(left)
    right ← MERGE SORT DESCENDING(right)
    merge the two halves by taking the larger next value
    RETURN merged list
```

Python code — core functions

```
def bubble_sort_desc(values, trace=False):
    a = list(values); states = []
    for p in range(len(a) - 1):
        swapped = False
        for j in range(len(a) - 1 - p):
            if a[j] < a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if trace and p < 3: states.append(a.copy())
        if not swapped: break
    return (a, states) if trace else a

def merge_sort_desc(a):
    if len(a) <= 1: return list(a)
    mid = len(a) // 2
    left = merge_sort_desc(a[:mid]); right = merge_sort_desc(a[mid:])
    out = []; i = j = 0
    while i < len(left) and j < len(right):
        if left[i] >= right[j]: out.append(left[i]); i += 1
        else: out.append(right[j]); j += 1
    return out + left[i:] + right[j:]

built_in = sorted(values, reverse=True)
```

Sample run — Question 2

I sorted the ten sample totals in descending order. The console output shows the first three passes and the benchmark on 10,000 real totals.

Program output

```
TASK 2 — BUBBLE SORT AND MERGE SORT
Sample: 10 real delivered Olist order totals; sort descending.
Start (cents): 1797, 14146, 4944, 17912, 1797, 4944, 5111, 5111, 5111, 3871
After pass 1 (cents): 14146, 4944, 17912, 1797, 4944, 5111, 5111, 5111, 3871, 1797
After pass 2 (cents): 14146, 17912, 4944, 4944, 5111, 5111, 5111, 3871, 1797, 1797
After pass 3 (cents): 17912, 14146, 4944, 5111, 5111, 5111, 4944, 3871, 1797, 1797
Final BRL order: R$179.12, R$141.46, R$51.11, R$51.11, R$51.11, R$49.44, R$49.44, R$38.71, R$17.97, R$17.97
10,000 actual delivered order totals; outputs identical: True
Bubble sort median: 5.8772 s
Merge sort median: 0.0252 s
Python sorted median: 0.0015 s
Bubble O(n^2) time, O(1) space; merge O(n log n), O(n) extra.
```

Sorted sample totals

R$179.12, R$141.46, R$51.11, R$51.11, R$51.11, R$49.44, R$49.44, R$38.71, R$17.97, R$17.97.

Bubble sort, merge sort and Python sorted() return the same descending order.

Dry run: bubble sort

Bubble sort compares neighboring amounts from left to right and swaps when the left value is smaller. This table shows the entire ten-item array after each of the first three passes. Values are cents.

State	0	1	2	3	4	5	6	7	8	9
Start	1797	14146	4944	17912	1797	4944	5111	5111	5111	3871
Pass 1	14146	4944	17912	1797	4944	5111	5111	5111	3871	1797
Pass 2	14146	17912	4944	4944	5111	5111	5111	3871	1797	1797
Pass 3	17912	14146	4944	5111	5111	5111	4944	3871	1797	1797

Each entry is an amount in cents: 1,797 = R$17.97, 14,146 = R$141.46, and so on. The trace demonstrates the complete array state—not only the already-sorted prefix.

Merge-sort trace on a four-value slice

Step	Values (cents)
Input	1,797 | 14,146 | 4,944 | 17,912
Split	[1,797, 14,146] | [4,944, 17,912]
Sort halves	[14,146, 1,797] | [17,912, 4,944]
Merge descending	[17,912, 14,146, 4,944, 1,797]

The merge compares the next item from each sorted half and appends the larger one until one side is exhausted.

Results — Question 2

I timed all three methods on the same 10,000 real order totals. Each sorted result was checked against Python sorted().

Method	Median time	Time complexity	Extra space
Bubble sort	5.872 s	$O(n^2)$	O(1)
Merge sort	0.0252 s	O(n log n)	O(n)
Python sorted / Timsort	0.0015 s	O(n log n) worst case	O(n) temporary

Complexity

Bubble sort is $O(n^2)$ average/worst and O(1) auxiliary space. Merge sort is O(n log n) time and O(n) auxiliary space. Python's built-in sort is highly optimized and can exploit existing runs in the data.

Question 3

Task 3 – Duplicate order check with a hash map

Aim: Find orders with the same customerId, amount, and purchase minute. Store matching order IDs in a hash map.

Pseudocode

```
FIND MATCHING GROUPS(orders)
    groups ← EMPTY HASH MAP
    FOR each order in orders
        amountCents ← exact BRL amount converted to cents
        minute ← createdAt through the minute (drop seconds)
        key ← (customerId, amountCents, minute)
        APPEND order.orderId to groups[key]
    RETURN groups with at least two order IDs
```

Python code — core function

```
from collections import defaultdict
from decimal import Decimal

def duplicate_groups(orders):
    groups = defaultdict(list)
    for order in orders:
        cents = int(Decimal(order["amount"]) * 100)
        minute = order["createdAt"][:16]
        key = (order["customerId"], cents, minute)
        groups[key].append(order["orderId"])
    return {key: ids for key, ids in groups.items() if len(ids) > 1}
```

Rule applied

Key component	Value taken from data	Comparison rule
Customer	stable customer_unique_id	Exact string match
Amount	order total in cents	Exact integer match
Purchase minute	createdAt[:16]	Year-month-day hour:minute; seconds ignored

Sample run — Question 3

The ten sample orders produce three matching groups, containing seven order IDs in total.

Program output

```
TASK 3 — HASH DUPLICATE REVIEW
Key: stable customerId + amount in cents + purchase minute.
Ten actual orders: 3 matching groups; 7 orders flagged.
Group 1: customer 031e19fc630c4121f1238716f41675c3; R$17.97; 2018-07-04 01:20
  Matching order IDs:
    246b14cc444e8a7df0afd80495dfa052
    f707ff2a9514dce20b267136b60af042
Group 2: customer 050555c26479bbb8ba0740c37e7454d0; R$49.44; 2018-04-07 12:07
  Matching order IDs:
    f014aa85d8f495d824d120e9035faacb
    4f173306347cca2ee2a05590d617854a
Group 3: customer 06a52782a04f0086d16b9c22d0e29438; R$51.11; 2017-05-06 20:11
  Matching order IDs:
    56ae029ed8bd31758f66831a64bf5629
    d9e44c3fd2ce16086619f299e92e12d8
    84f5e6c0a0e3155e38c00f434ba90ce8
Flags indicate review under the rule; they do not prove erroneous submissions.
Expected time O(n); auxiliary hash-map space O(n).
```

Screenshot: output from the duplicate-check program.

Matching groups in the sample

Group	customerId	amount	minute	orderId values
Group 1	`031e19fc630c4121f1238716f41675c3`	R$17.97	2018-07-04 01:20	`246b14cc444e8a7df0afd80495dfa052` `f707ff2a9514dce20b267136b60af042`
Group 2	`050555c26479bbb8ba0740c37e7454d0`	R$49.44	2018-04-07 12:07	`f014aa85d8f495d824d120e9035faacb` `4f173306347cca2ee2a05590d617854a`

Group 3	`06a52782a04f0086d16b9c22d0e29438`	R$51.11	2017-05-06 20:11	`56ae029ed8bd31758f66831a64bf5629`
				`d9e44c3fd2ce16086619f299e92e12d8`
				`84f5e6c0a0e3155e38c00f434ba90ce8`

Dry run: hash-map grouping

Read the rows in order. K1–K3 are the repeated keys shown on the previous page; U1–U3 appear once. A new key starts a bucket. A matching key adds another order ID to that bucket.

Input row	ID suffix	Key	IDs stored after insert	Hash-map action
1	`95dfa052`	K1	1	new bucket
2	`50273451`	U1	1	new bucket
3	`035faacb`	K2	1	new bucket
4	`f07ec65d`	U2	1	new bucket
5	`b60af042`	K1	2	match; bucket now 2
6	`d617854a`	K2	2	match; bucket now 2
7	`64bf5629`	K3	1	new bucket
8	`e92e12d8`	K3	2	match; bucket now 2
9	`4ba90ce8`	K3	3	match; bucket now 3
10	`6f2d6af7`	U3	1	new bucket

Result and complexity

Three keys have more than one ID (group sizes 2, 2, and 3), so seven orders are flagged for review. Building the map takes expected O(n) time and O(n) space. A Python dictionary lookup is expected O(1).

Question 4

Task 4 – Undo stack and dispatch queue

Aim: Use a stack to undo the last edit and a queue to process orders in dispatch order. Compare the custom structures with list and deque.

Pseudocode

```
EDIT ORDER(order, field, newValue, undoStack)
    PUSH (order.orderId, field, order[field])  // save OLD value first
    order[field] ← newValue

UNDO LAST EDIT(undoStack, order)
    IF undoStack is empty: RETURN NO_ACTION
    (orderId, field, oldValue) ← POP undoStack
    restore order[field] ← oldValue

ENQUEUE(queue, orderId): append at the tail
 DEQUEUE(queue): return item at head; advance head
```

Python code — own structures and use

```
from collections import deque
class Stack:
    def __init__(self): self.items = []
    def push(self, x): self.items.append(x)
    def pop(self):
        if not self.items: raise IndexError("empty stack")
        return self.items.pop()
class Queue:
    def __init__(self): self.items, self.head = [], 0
    def enqueue(self, x): self.items.append(x)
    def dequeue(self):
        if self.head == len(self.items): raise IndexError("empty queue")
        x = self.items[self.head]; self.head += 1; return x

# Demonstration: the sample row starts at 1,797 cents (R$17.97)
order_id = sample[0]["orderId"]; amount = 1797
history = Stack()
for new_amount in (1897, 2047):
    history.push((order_id, "amount", amount))  # save BEFORE edit
    amount = new_amount
_, field, old_amount = history.pop(); amount = old_amount
custom_restored = amount  # one undo restores 1,897 cents

order_ids = [r["orderId"] for r in sample[:3]]
q = Queue()
for oid in order_ids: q.enqueue(oid)
custom_order = [q.dequeue() for _ in order_ids]

# Built-in equivalents: list for stack, deque for FIFO queue
b_amount = 1797; undo_list = []
for new_amount in (1897, 2047):
    undo_list.append((order_id, "amount", b_amount)); b_amount = new_amount
_, _, prior = undo_list.pop(); b_amount = prior
bq = deque(order_ids)
builtin_order = [bq.popleft() for _ in order_ids]
assert custom_restored == b_amount and custom_order == builtin_order
```

The custom queue's head index avoids removing index 0 from a list (which would shift all remaining elements).

Sample run — Question 4

The example changes one real sample amount twice, then undoes the last change. Each old amount is saved before the edit.

Program output

```
TASK 4 — STACK UNDO AND FIFO DISPATCH
Real sample order: 246b14cc444e8a7df0afd80495dfa052
Initial amount: R$17.97
Edit 1: R$17.97 -> R$18.97; saved previous value R$17.97.
Edit 2: R$18.97 -> R$20.47; saved previous value R$18.97.
Undo once: restored R$18.97; remaining stack depth 1.
Built-in list undo agrees: True.
Queue IDs: 246b14cc444e8a7df0afd80495dfa052 -> 53cdb2fc8bc7dce0b6741e2150273451 -> f014aa85d8f495d824d120e9035faacb
Custom queue dequeue order: ['246b14cc444e8a7df0afd80495dfa052', '53cdb2fc8bc7dce0b6741e2150273451', 'f014aa85d8f49
5d824d120e9035faacb']
collections.deque order matches: True
Stack push/pop O(1) amortized; head-index queue enqueue/dequeue O(1) amortized.
The edit/dispatch events are demonstrations, not source-data event records.
```

Screenshot: output from the stack and queue program.

Dry run: stack and queue

Stack dry run — state after each action

Action	Amount now	Stack (bottom → top)
Initial state	R$17.97	[]
Edit 1: save 17.97, set 18.97	R$18.97	[(order, R$17.97)]
Edit 2: save 18.97, set 20.47	R$20.47	[(order, R$17.97), (order, R$18.97)]
Undo: pop the top saved value	R$18.97	[(order, R$17.97)]

The demonstrated order is 246b14cc444e8a7df0afd80495dfa052. The last saved amount (R$18.97) is on top, so undo restores R$18.97 and leaves the earlier snapshot available for another undo.

Queue dry run — first three real IDs

Operation	Queue contents / returned order
Enqueue 1	`[246b14cc444e8a7df0afd80495dfa052]`
Enqueue 2	`[246b14cc444e8a7df0afd80495dfa052, 53cdb2fc8bc7dce0b6741e2150273451]`
Enqueue 3	`[246b14cc444e8a7df0afd80495dfa052, 53cdb2fc8bc7dce0b6741e2150273451, f014aa85d8f495d824d120e9035faacb]`
Dequeue × 3	`246b14cc444e8a7df0afd80495dfa052 → 53cdb2fc8bc7dce0b6741e2150273451 → f014aa85d8f495d824d120e9035faacb`

Complexity

Stack push and pop take O(1) time on average. The head-index queue and collections.deque also process each item in O(1) amortized time. Both structures use O(n) storage.

Question 5

Task 5 – Refund change with greedy and dynamic programming

Aim: Find the fewest notes or coins for an amount. Show a case where greedy is not optimal, then solve it with dynamic programming.

Pseudocode

```
GREEDY CHANGE(amount, denominations)
    sort denominations from largest to smallest
    FOR each denomination d
        take as many d as fit; subtract their value
    IF remainder is zero: RETURN chosen pieces
    ELSE: RETURN no exact solution

MINIMUM CHANGE(amount, denominations)
    best[0] ← 0; best[1..amount] ← infinity
    FOR value ← 1 TO amount
        FOR each denomination d ≤ value
            best[value] ← MIN(best[value], best[value – d] + 1)
            remember d when it gives a better count
    follow remembered choices from amount down to zero
    RETURN chosen pieces
```

Python code — core functions

```
def greedy_change(amount, denominations):
    result = []
    for coin in sorted(denominations, reverse=True):
        while amount >= coin:
            result.append(coin); amount -= coin
    return result if amount == 0 else None

def min_change(amount, denominations):
    best = [0] + [float("inf")] * amount
    last = [None] * (amount + 1)
    for value in range(1, amount + 1):
        for coin in denominations:
            if coin <= value and best[value-coin] + 1 < best[value]:
                best[value] = best[value-coin] + 1; last[value] = coin
    if best[amount] == float("inf"): return None
    result = []
    while amount:
        coin = last[amount]; result.append(coin); amount -= coin
    return result
```

Sample run — Question 5

I used the R$51.11 total from orderId 56ae029ed8bd31758f66831a64bf5629 as a hypothetical refund amount. Greedy and DP each return four BRL pieces.

Program output

```
TASK 5 – GREEDY CHANGE AND DYNAMIC PROGRAMMING
Actual Olist amount used only as a hypothetical refund value.
Order 56ae029ed8bd31758f66831a64bf5629: R$51.11.
Brazilian denominations, greedy pieces: 5000, 100, 10, 1
BRL greedy pieces: 4; DP pieces: 4; same minimum: True.
Non-canonical abstract denominations [1, 4, 5], amount 8:
Greedy: [5, 1, 1, 1] (4 pieces); DP: [4, 4] (2 pieces).
This abstract set is a counterexample, not a real currency system.
DP complexity O(Ak) time and O(A) auxiliary space.
```

Screenshot: output from the change-making program.

Results

Case	Greedy result	DP result	Meaning
R$51.11; BRL denominations	R$50 + R$1 + R$0.10 + R$0.01 (4 pieces)	Same four pieces	Greedy is minimum for this example
Abstract [1, 4, 5]; amount 8	5 + 1 + 1 + 1 (4 pieces)	4 + 4 (2 pieces)	Greedy is not generally optimal

Dry run: change-making

Greedy dry run — 5,111 cents

The 10/5/1-cent steps below are the denominations actually selected; larger denominations are skipped when they exceed the remaining amount.

Remaining before	Denomination taken	Count	Remaining after
5,111	5,000	1	111
111	100	1	11
11	10	1	1
1	1	1	0

Dynamic-programming dry run — abstract denominations [1, 4, 5], amount 8

x	Candidate count (coin: pieces)	best[x]	coin chosen
0	—	0	—
1	1: 1	1	1
2	1: 2	2	1
3	1: 3	3	1
4	1: 4, 4: 1	1	4
5	1: 2, 4: 2, 5: 1	1	5
6	1: 2, 4: 3, 5: 2	2	1
7	1: 3, 4: 4, 5: 3	3	1
8	1: 4, 4: 2, 5: 4	2	4

At x = 8, coin 4 gives best[4] + 1 = 2 pieces; two 4s make 8. Greedy instead takes 5, then three 1s: four pieces.

Result and complexity

For R$51.11, greedy and DP both use four pieces. Greedy sorts the denominations, then takes the largest coin that fits. DP takes O(Ak) time and O(A) space, where A is cents and k is the number of denominations.

Complexity summary and sources

Task	Algorithm / structure	Time	Extra space
1	Linear search	O(n)	O(1)
1	Binary search (sorted IDs)	O(log n)	O(1)
1	Build sorted index	O(n log n)	O(n)
2	Bubble sort	$O(n^2)$	O(1)
2	Merge sort	O(n log n)	O(n)
2	Python sorted()	O(n log n) worst case	O(n) temporary
3	Hash grouping	O(n) expected	O(n)
4	Stack push / pop	O(1) amortized	O(n) storage
4	Head-index queue / deque	O(1) amortized	O(n) storage
5	Dynamic-programming change	O(Ak)	O(A)